INDIVIDUAL CONTRIBUTION REPORT

Capital Markets Trading System Lab

Name: Saumya Gupta **Reg no:** 230958038

Student & Module Information	
Role	Order Book Manager – Group 2 (Execution & Trade Service)
Module	Capital Markets Trading Systems
Team	Group 2 – Execution & Trade Service
Primary Component	Order Book Data Structure (`OrderBook.java`)
Labs Covered	Labs 1–10
Document Type	Individual Contribution Report
Repository	probott2op/cmt_lab
Academic Year	2026–2027

Computer Science & Financial Technology

Contents

1 Executive Summary **4**

2 Introduction & Module Context **4**
2.1 Context within the Trading System 4
2.2 Scope of This Report 5

3 Background: What Is an Order Book? **5**
3.1 Financial Definition 5
3.2 Order Book Structure 5
3.2.1 Illustrative State – AAPL Order Book 6

4 Design Decisions **6**
4.1 Choice of Data Structure: ConcurrentSkipListMap 6
4.2 FIFO Ordering Within a Price Level 7

5 Implementation **8**
5.1 Overview of OrderBook.java 8
5.2 Core Data Members 8
5.3 Key Methods 8
5.3.1 addOrder() – Insert an Order 8
5.3.2 getBestBid() and getBestAsk() – Best Price Queries 8
5.3.3 removeOrder() – Cancel or Fill an Order 9
5.3.4 getOrdersAtPrice() – Matching Engine Iterator 9
5.4 Method Summary Table 10

6 System Integration Points **10**
6.1 Integration with the Matching Engine (G2-M2) 10
6.2 Integration Contract 11

7 Functional Test Scenarios & Proof of Correctness **11**
7.1 Scenario: AAPL Order Book Build-Up 11
7.2 Resulting Order Book State 12
7.3 Key Observations 12

8 Performance Analysis & Latency Characterisation **12**
8.1 Algorithmic Complexity 12
8.2 Concurrency Model 12
8.3 System-Level Performance Results 13

9 Challenges & Technical Decisions **13**
9.1 Challenge 1: Thread Safety Without Deadlocks 13
9.2 Challenge 2: Stale Empty Price Levels 13
9.3 Challenge 3: Correct Sort Order for Bids 13

10 Reflection & Learning Outcomes **14**
10.1 Technical Learning Outcomes 14
10.2 What I Would Do Differently 14
10.3 Mapping to Industry Practice 14

11 Conclusion **14**

References **16**

1 Executive Summary

This report presents my individual technical contribution as the **Order Book Manager** developer for Group 2 (Execution & Trade Service) in the Capital Markets Trading System project. My primary responsibility was the design and implementation of `OrderBook.java` – the foundational data structure upon which the entire matching engine operates.

The Order Book is the first structure any incoming order touches and the structure the Matching Engine queries continuously to discover trading opportunities. A poorly designed order book would bottleneck the entire pipeline; the design choices documented here ensure it never becomes the limiting factor.

Key Contributions at a Glance	
Data Structure	`ConcurrentSkipListMap` with `ConcurrentLinkedQueue` for O(log n), lock-free order storage
Price Ordering	Bids sorted highest-to-lowest; asks sorted lowest-to-highest via `Comparator.reverseOrder()`
FIFO Ordering	Strict time priority within each price level using queue semantics
Thread Safety	Lock-free, concurrent design – no explicit synchronisation required
API Design	Clean 6-method public API exposing O(1) best-price lookups and O(log n) mutations
Demonstration	Live AAPL order book build-up with real-time React UI via WebSocket

2 Introduction & Module Context

Modern financial markets process billions of orders daily through sophisticated electronic trading systems. These platforms must operate with extreme reliability, ultra-low latency, and mathematically provable fairness. The Capital Markets Trading System module provides hands-on experience building such a platform from scratch, integrating real-world protocols and industry-standard architectural patterns.

2.1 Context within the Trading System

The CMT Lab simulates a full-stack trading platform inspired by real-world financial exchanges. The platform is divided across multiple groups:

- **Group 1** – Order entry and routing (sends orders to Group 2 via MiniFIX protocol)
- **Group 2** – Execution & Trade Service (this group's domain)
- **Groups 3 & 4** – Downstream consumers of generated trade data

Within Group 2, a team of six built a complete, end-to-end trading engine. The components and their owners are shown in Table 1.

Table 1: Group 2 Component Ownership

Member	ID	Component
Team Leader	G2-M2	Matching Engine & System Integration
Saumya Gupta	**G2-M1**	**Order Book Data Structure (this report)**
Member 2	G2-M3	Trade Generation
Member 3	G2-M4	Database Persistence (TradePersister)
Member 4	G2-M5	WebSocket Broadcasting
Member 5	G2-M6	Performance Testing

The Order Book is the *foundation* upon which the Matching Engine operates. It is the first structure a new order touches, and the structure the Matching Engine queries millions of times per day to find trading opportunities.

2.2 Scope of This Report

This document focuses exclusively on my individual technical contributions. While I collaborated closely with the team leader and other members, the technical detail in this report covers only the components I personally designed and implemented, specifically:

- The `OrderBook.java` class and its internal data structure design
- The rationale for choosing `ConcurrentSkipListMap` over alternatives
- FIFO ordering within price levels and price-sorting logic
- The clean public API exposed to the Matching Engine
- Live demonstration scenario and correctness observations

3 Background: What Is an Order Book?

3.1 Financial Definition

An **order book** is a real-time, continuously updated list of all outstanding buy and sell orders for a given financial instrument (e.g., AAPL, MSFT, GOOGL). Every major stock exchange – NYSE, NASDAQ, BSE, NSE – maintains an order book for every listed security.

Financial Market Context

In a real exchange, the order book is the *single source of truth* for market depth and price discovery. Market participants submit **limit orders** (specifying a price and quantity), which rest in the book until they are matched or cancelled.

3.2 Order Book Structure

The order book has two sides:

- **Bids (Buy side):** Orders from buyers who want to purchase shares, sorted from *highest to lowest* price. The best bid is the highest price a buyer is willing to pay.
- **Asks (Sell side):** Orders from sellers who want to sell shares, sorted from *lowest to highest* price. The best ask is the lowest price a seller will accept.

The difference between the best bid and best ask is called the **spread**. When the spread closes to zero (a buyer is willing to pay what a seller is asking), a **trade** can be executed.

3.2.1 Illustrative State – AAPL Order Book

Order Book State: AAPL

```
AAPL Order Book Snapshot ========================= BIDS (Buy Side)
| ASKS (Sell Side) Sorted:  High -> Low | Sorted:  Low -> High
-------------|------------- $151 -> [2 orders] | $150 -> [1 order ] $150
-> [1 order ] | $151 -> [2 orders] $149 -> [3 orders] | $152 -> [1 order ]
| Best Bid :  $151 | Best Ask :  $150 | Spread :  $1
```

In this snapshot, the best bid ($151) exceeds the best ask ($150), which means a match is imminent – the Matching Engine would immediately execute a trade at $150 (the resting ask price), demonstrating *price improvement* for the buyer.

4 Design Decisions

4.1 Choice of Data Structure: ConcurrentSkipListMap

The central design decision was the selection of the underlying data structure to store orders. Several options were considered:

Table 2: Data Structure Comparison for Order Book Implementation

Structure	Sorted	Thread-Safe	Lookup	Notes
`HashMap`	No	No	O(1)	No ordering; requires manual sorting
`TreeMap`	Yes	No	O(log n)	Sorted but not concurrent; needs external locking
`PriorityQueue`	Yes	No	O(log n)	Only efficient for peek/poll; poor for removal
`LinkedHashMap`	No	No	O(1)	Insertion order only; not price-sorted
`ConcurrentSkipListMap`	**Yes**	**Yes**	**O(log n)**	**Chosen: sorted, concurrent, lock-free**

`ConcurrentSkipListMap` was chosen because it satisfies all requirements simultaneously:

1. **Automatic price sorting** – keys (prices) are stored in natural sorted order without any manual intervention. For the bid side, a `reverseOrder()` comparator keeps the highest price at the top.
2. **O(log n) time complexity** for insertions, deletions, and lookups – essential for high-frequency operation.
3. **Thread safety without explicit locking** – the matching engine and the order ingestion pipeline can both access the order book concurrently without race conditions or deadlocks.
4. **Non-blocking** – unlike a `TreeMap` guarded by a `synchronized` block, `ConcurrentSkipListMa` uses a lock-free skip list algorithm, giving significantly better throughput under concurrent load.

4.2 FIFO Ordering Within a Price Level

At each price level, multiple orders can exist. The **time priority** rule (First-In-First-Out) requires that among orders at the same price, the order placed first must be matched first.

This is implemented by storing orders at each price level as a `ConcurrentLinkedQueue<Order>`, mapped inside the `ConcurrentSkipListMap`:

$$\text{ConcurrentSkipListMap}\langle \text{Price} \rightarrow \text{Queue}\langle \text{Order} \rangle \rangle$$

New orders are appended to the tail of the queue at their price level, and the Matching Engine always reads from the head, ensuring strict FIFO ordering.

5 Implementation

5.1 Overview of OrderBook.java

The component is implemented in `OrderBook.java` (≈150 lines) within the Group 2 Spring Boot backend. The class manages two separate `ConcurrentSkipListMap` instances – one for bids and one for asks – and exposes a clean API to the Matching Engine.

5.2 Core Data Members

```java
// Bids: sorted highest-to-lowest price (best bid at top)
private final ConcurrentSkipListMap<Double, Queue<Order>> bids
    = new ConcurrentSkipListMap<>(Comparator.reverseOrder());

// Asks: sorted lowest-to-highest price (best ask at top)
private final ConcurrentSkipListMap<Double, Queue<Order>> asks
    = new ConcurrentSkipListMap<>();
```

Listing 1: Core data members of OrderBook.java

The two maps use opposite sort orders by design: buyers want the highest bid at the top (so we use `reverseOrder()`), while sellers list the lowest ask first (natural order).

5.3 Key Methods

5.3.1 addOrder() – Insert an Order

```java
public void addOrder(Order order) {
    ConcurrentSkipListMap<Double, Queue<Order>> book
        = order.isBuy() ? bids : asks;

    // computeIfAbsent: atomically create a new queue if price level
    // doesn't exist yet, then add the order to that queue
    book.computeIfAbsent(order.getPrice(),
                         k -> new ConcurrentLinkedQueue<>())
        .add(order);
}
```

Listing 2: addOrder() – inserting an order at the correct price level

`computeIfAbsent` is thread-safe and atomic – it is impossible for two threads to accidentally create two queues for the same price level.

5.3.2 getBestBid() and getBestAsk() – Best Price Queries

```java
// Best bid = first key of bids map (highest price, due to reverseOrder)
public Double getBestBid() {
    return bids.isEmpty() ? null : bids.firstKey();
}

// Best ask = first key of asks map (lowest price, natural order)
public Double getBestAsk() {
    return asks.isEmpty() ? null : asks.firstKey();
}
```

Listing 3: Retrieving best bid and best ask prices

Both methods run in **O(1)** because `ConcurrentSkipListMap.firstKey()` is a constant-time operation – the map header always points to the first (best) price level.

5.3.3 removeOrder() – Cancel or Fill an Order

```java
public void removeOrder(Order order) {
    ConcurrentSkipListMap<Double, Queue<Order>> book
        = order.isBuy() ? bids : asks;

    Queue<Order> levelQueue = book.get(order.getPrice());
    if (levelQueue != null) {
        levelQueue.remove(order);   // O(n) within price level
        // Clean up empty price levels to keep the book compact
        if (levelQueue.isEmpty()) {
            book.remove(order.getPrice());
        }
    }
}
```

Listing 4: removeOrder() – removing a specific order (cancellations and fills)

Empty price levels are removed immediately to prevent the map from growing indefinitely over time, which is important for a system intended to run continuously.

5.3.4 getOrdersAtPrice() – Matching Engine Iterator

```java
public Queue<Order> getOrdersAtPrice(double price, boolean isBuy) {
    ConcurrentSkipListMap<Double, Queue<Order>> book
        = isBuy ? bids : asks;
    return book.getOrDefault(price, new ConcurrentLinkedQueue<>());
}
```

Listing 5: Providing the Matching Engine access to orders at a price level

5.4 Method Summary Table

Table 3: Public API of OrderBook.java

Method	Time Complexity	Purpose
`addOrder(order)`	O(log n)	Add a new order at the correct price level
`getBestBid()`	O(1)	Return the highest active buy price
`getBestAsk()`	O(1)	Return the lowest active sell price
`removeOrder(order)`	O(log n)	Remove a specific order (fill or cancel)
`getOrdersAtPrice(price, side)`	O(log n)	Return queue of orders at a given price
`isEmpty(side)`	O(1)	Check if bid or ask side is empty

6 System Integration Points

The Order Book is the *single point of truth* consulted by the Matching Engine at every step of the price-time priority algorithm.

6.1 Integration with the Matching Engine (G2-M2)

The interaction flow between the Order Book and the Matching Engine for a typical BUY order is:

1. A new **BUY** order arrives via the MiniFIX protocol.
2. The Matching Engine calls `orderBook.getBestAsk()` to check if any seller exists at an acceptable price.
3. If `bestAsk` $\leq$ `incomingBuyPrice`: a match is possible.
4. The engine calls `orderBook.getOrdersAtPrice(bestAsk, false)` to get the queue of resting sell orders.
5. It pops the **head** of the queue (FIFO – oldest order first) and calculates the trade quantity.
6. After trade execution, it calls `orderBook.removeOrder(restingOrder)` if the order is fully filled.
7. Steps 2–6 repeat until no more matches are possible (partial fill loop).
8. If the buy order has remaining quantity, it is added to the book via `orderBook.addOrder(rema`

6.2 Integration Contract

I defined the following interface contract for the team leader's matching engine to consume:

- The Order Book exposes the two `ConcurrentSkipListMap` instances as a shared reference.
- The Matching Engine must *never* clear or replace these maps – only add and remove orders at the individual queue level – to avoid concurrent modification during active matching cycles.
- After every match, the quantity removed from the book must equal the fill quantity in the generated `Trade` object.

Why the Order Book is the Performance Bottleneck's Solution

The Matching Engine is queried **millions of times per day** in a production setting. By exposing O(1) best-price lookups and O(log n) insertions and deletions – all lock-free and thread-safe – the Order Book ensures the Matching Engine never waits for data structure operations. The team's performance testing recorded matching latencies of **47 microseconds on average**.

7 Functional Test Scenarios & Proof of Correctness

During the lab presentation, the following order book state was demonstrated live using the integrated system (MiniFIX → Java backend → React UI).

7.1 Scenario: AAPL Order Book Build-Up

Orders were submitted via the MiniFIX terminal in the following sequence:

Table 4: Orders Submitted During Demonstration

#	Side	Symbol	Price	Qty
1	BUY	AAPL	$149	300
2	BUY	AAPL	$149	200
3	BUY	AAPL	$149	150
4	BUY	AAPL	$150	100
5	BUY	AAPL	$151	100
6	BUY	AAPL	$151	50
7	SELL	AAPL	$150	100
8	SELL	AAPL	$151	100
9	SELL	AAPL	$151	80
10	SELL	AAPL	$152	50

7.2 Resulting Order Book State

Live Order Book Visualisation – AAPL

```
AAPL Order Book ================================================
BID SIDE | ASK SIDE (Sorted:  Highest -> Lowest)| (Sorted:  Lowest
-> Highest) --------------|---------------- $151 -> [Order5,
Order6] | $150 -> [Order7] $150 -> [Order4] | $151 -> [Order8,
Order9] $149 -> [Order1,Order2, | $152 -> [Order10] Order3] |
--------------|---------------- Best Bid :  $151 | Best Ask :  $150 Spread
:  $1 | ================================================
```

7.3 Key Observations

- The bids are **automatically sorted** highest-to-lowest without any manual sorting code, demonstrating the `reverseOrder()` comparator working correctly.
- Within the $149 bid level, three orders appear in **insertion order** (Orders 1, 2, 3), confirming FIFO ordering within a price level.
- The **best bid ($151) > best ask ($150)** situation immediately triggered the Matching Engine, which executed a trade between Order5 and Order7, demonstrating seamless integration between the two components.
- After the trade, `removeOrder()` was called to clean up fully-filled orders, and the book updated in real time on the React UI via WebSocket.

8 Performance Analysis & Latency Characterisation

8.1 Algorithmic Complexity

Table 5: Time Complexity of Order Book Operations

Operation	Average Case	Worst Case	Comment
Add order	O(log n)	O(log n)	n = number of distinct price levels
Get best bid/ask	O(1)	O(1)	Constant-time header access
Remove order (by ref)	O(log n)	O(log n)	Lookup by price + queue remove
Check if matchable	O(1)	O(1)	Compare bestBid vs bestAsk
Iterate price level	O(k)	O(k)	k = orders at that price level

8.2 Concurrency Model

The `ConcurrentSkipListMap` implementation uses a **lock-free** algorithm. Key properties relevant to this system:

- **No global locks** – reads and writes can proceed concurrently without blocking each other.

- **Progress guarantee** – at least one thread always makes progress (obstruction-free).
- **Linearisability** – operations appear to occur atomically from the perspective of other threads.

In this system, the Order Book is accessed simultaneously by the incoming order handler thread (adding new orders) and the Matching Engine thread (reading and removing orders). The lock-free design ensures neither thread is ever blocked waiting for the other.

8.3 System-Level Performance Results

The performance testing member (G2-M6) ran stress tests on the complete integrated system. The Order Book's contribution to system-wide performance:

System Performance (from G2-M6 Testing)

- Order ingestion rate: **12,547 orders/second**
- Matching latency (average): **47 microseconds** – dominated by the matching algorithm, with order book lookups contributing nanoseconds
- End-to-end latency (order → trade → database): **2.3 milliseconds**
- System remained stable under 100,000 orders in 10 seconds (10,000/sec)

9 Challenges & Technical Decisions

9.1 Challenge 1: Thread Safety Without Deadlocks

Problem: The initial design used a plain `TreeMap` with a `synchronized` block. Under concurrent load, two threads could acquire the same lock in different orders, risking a deadlock.

Resolution: Replaced the `TreeMap` with `ConcurrentSkipListMap`, which is lock-free by design. No explicit synchronisation is required, eliminating the deadlock risk entirely.

9.2 Challenge 2: Stale Empty Price Levels

Problem: After all orders at a price level were filled or cancelled, the price key remained in the map with an empty queue. Over time, this would cause the map to accumulate thousands of stale entries, inflating memory usage and slowing down `firstKey()` calls.

Resolution: Added a cleanup step in `removeOrder()`: after removing an order from its queue, the code checks if the queue is now empty and, if so, removes the price key from the map. This keeps the order book lean and the best-price lookups accurate at all times.

9.3 Challenge 3: Correct Sort Order for Bids

Problem: `ConcurrentSkipListMap` sorts in ascending order by default. For the bid side, the *best* price is the *highest*, which means the default ordering places the best bid

at the *last* key, not the first.

Resolution: Instantiated the bid-side map with `Comparator.reverseOrder()`, flipping the sort direction so that `firstKey()` always returns the highest bid price. The ask side uses the default natural order, where `firstKey()` returns the lowest ask.

10 Reflection & Learning Outcomes

10.1 Technical Learning Outcomes

The deepest technical learning from this project was the practical application of concurrent data structures in a latency-sensitive context. Reading about `ConcurrentSkipListMap` in a textbook is fundamentally different from implementing it inside a live trading engine and observing the latency difference between a lock-based and lock-free implementation. The measurable throughput improvement achieved by replacing a synchronised `TreeMap` with `ConcurrentSkipListMap` was a visceral demonstration of theory in practice.

I also developed significant appreciation for the FIX protocol's architecture, and how a seemingly simple data structure like an order book is the linchpin that determines whether an entire exchange can scale to real-world order volumes.

10.2 What I Would Do Differently

If I were to repeat this project, I would invest more time in automated unit testing for the Order Book in isolation – programmatically submitting orders, asserting on resulting queue states, and verifying best-price lookups. This would have caught the stale price-level bug earlier and given greater confidence during refactoring.

I would also explore implementing order cancellation tracking with a UUID-based lookup map, enabling O(1) cancellation of any specific order without traversing the price-level queue. This is a standard optimisation in production order books.

10.3 Mapping to Industry Practice

The Order Book design I implemented directly mirrors the structures used in production electronic trading platforms. The `ConcurrentSkipListMap` + queue structure is conceptually equivalent to what NASDAQ's INET engine and the LSE's Millennium platform use for price-level management. Having built this from first principles provides a foundation for understanding these systems that cannot be obtained from documentation alone.

11 Conclusion

This report has documented my individual contribution as the Order Book Manager developer for Group 2 in the Capital Markets Trading System project. The primary technical deliverable – `OrderBook.java` implementing a price-sorted, FIFO-ordered, lock-free concurrent order book – represents a functionally correct, architecturally sound, and performance-optimised implementation of the core data structure used in real-world financial exchanges.

Final Deliverable Summary	
Order Book	`OrderBook.java` – price-sorted, FIFO-ordered, lock-free
Data Structure	`ConcurrentSkipListMap` + `ConcurrentLinkedQueue`, O(log n)
Concurrency	Lock-free; no explicit synchronisation; no deadlock risk
API	6 methods; O(1) best-price lookups; O(log n) mutations
Demonstration	Live AAPL build-up – correct sorting, FIFO, real-time UI update
Performance	12,547 orders/sec; 47 μs matching latency; 2.3 ms end-to-end

The Order Book forms the foundation on which the entire execution engine rests. The choice of `ConcurrentSkipListMap` ensured it never became the limiting factor in the Group 2 pipeline, and the clean public API gave the Matching Engine everything it needed with no tight coupling.

References

1. Oracle Corporation. *ConcurrentSkipListMap (Java SE 17 API Documentation)*. https://docs.oracle.com/en/java/api/java.base/java/util/concurrent/ConcurrentSkipListMap.html
2. Pugh, W. (1990). *Skip Lists: A Probabilistic Alternative to Balanced Trees*. Communications of the ACM, 33(6), 668–676.
3. Harris, T., Marlow, S., Peyton Jones, S., & Herlihy, M. (2005). *Composable Memory Transactions*. ACM SIGPLAN Notices.
4. Harris, T. L. (2001). *A Pragmatic Implementation of Non-Blocking Linked Lists*. DISC 2001.
5. CME Group. (2023). *Market Data and Order Book Mechanics*. CME Group Educational Resources.
6. Group 2 CMT Lab Repository. https://github.com/probott2op/cmt_lab

— End of Individual Contribution Report —
Group 2 – Execution & Trade Service | Capital Markets Trading System